

ABES Engineering College, Ghaziabad
 Department of Computer Science and Engineering
25CS301 – Object Oriented Programming System
Unit 1 MCQ Test

Student Name:**Roll No.:****Section:** ☐ CSE-21 ☐ CSE-26 **Date:****Maximum Marks: 30 Questions: 30****Instructions:**

- Attempt all questions. Each question carries one mark.
- Select exactly one best answer for each question.
- Assume standard C++17 behaviour unless a question states otherwise.

Q1. Which statement best describes the shift from structured programming to object-oriented programming?

- A. ☐ Programs stop using functions completely.
- B. ☐ Data and the operations on that data are grouped into objects.
- C. ☐ Every variable must become global.
- D. ☐ Only selection statements are allowed.

Q2. In structured programming, the three basic control structures are:

- A. ☐ Sequence, selection and iteration
- B. ☐ Class, object and inheritance
- C. ☐ Compilation, linking and execution
- D. ☐ Input, output and storage

Q3. Which statement correctly distinguishes a **class** from an **object**?

- A. ☐ A class is an instance; an object is the blueprint.
- B. ☐ A class is a blueprint; an object is an instance of that class.
- C. ☐ A class stores only functions; an object stores only data.
- D. ☐ Class and object are two names for the same C++ keyword.

Q4. Consider:

```
class Student {
    int marks;
public:
    void show();
};
```

Which statement is correct?

- A. ☐ `marks` is public because `show()` is public.

- B. ☐ `marks` is private by default in a class.
- C. ☐ `show()` cannot access `marks`.
- D. ☐ `Student` cannot have both data and functions.

Q5. Fill in the missing code so that the member function is defined outside the class correctly.

```
class Box {
public:
    void display();
};

void _____ display() {
    // statements
}
```

- A. ☐ `Box.`
- B. ☐ `Box::`
- C. ☐ `::Box.`
- D. ☐ `Box->`

Q6. A global variable `int value = 10;` is hidden by a local variable `int value = 20;`. Which expression accesses the global variable?

- A. ☐ `global.value`
- B. ☐ `this->value`
- C. ☐ `::value`
- D. ☐ `value::`

Q7. Which line makes the following class definition syntactically complete?

```
class Account {
public:
    void show();
} // missing token here
```

- A. ☐ `:`
- B. ☐ `;`

- C. ☐ ,
D. ☐ ::

Q8. Which declaration correctly creates a reference variable bound to `x`?

```
int x = 25;
```

- A. ☐ `int ref = &x;`
B. ☐ `int &ref = x;`
C. ☐ `int *ref = x;`
D. ☐ `int &ref;`

Q9. What is the output?

```
int x = 10;
int y = 40;
int &r = x;
r = y;
cout << x << " " << y;
```

- A. ☐ 10 40
B. ☐ 40 40
C. ☐ 10 10
D. ☐ Compilation error because a reference is reassigned

Q10. Which function uses **call by value**?

- A. ☐ `void update(int x)`
B. ☐ `void update(int& x)`
C. ☐ `void update(int* x)`
D. ☐ `void update(const int& x)`

Q11. What is the output?

```
void change(int x) {
    x = 99;
}
int main() {
    int a = 5;
    change(a);
    cout << a;
}
```

- A. ☐ 5
B. ☐ 99
C. ☐ 0
D. ☐ Compilation error

Q12. Choose the correct statement to make the function modify the caller's variable without using pointers.

```
void increase( _____ ) {
    x++;
}
```

- A. ☐ `int x`
B. ☐ `int& x`
C. ☐ `const int& x`
D. ☐ `int* x`

Q13. Which call correctly passes the **address** of number to the function?

```
void reset(int *p);
int number = 8;
```

- A. ☐ `reset(number);`
B. ☐ `reset(*number);`
C. ☐ `reset(&number);`
D. ☐ `reset(&reset);`

Q14. Fill in the missing statement to make the pointer-based function safe for a null pointer.

```
bool setZero(int *p) {
    if ( _____ )
        return false;
    *p = 0;
    return true;
}
```

- A. ☐ `p == nullptr`
B. ☐ `*p == nullptr`
C. ☐ `&p == nullptr`
D. ☐ `p = nullptr`

Q15. Which is a valid function prototype for a function that receives two integers and returns their sum as an integer?

- A. ☐ `int add(int, int);`
B. ☐ `add(int, int) int;`
C. ☐ `int add = (int, int);`
D. ☐ `function int add(int, int);`

Q16. In the call `sum(a, b)`, `a` and `b` are called:

- A. ☐ formal parameters
B. ☐ actual parameters
C. ☐ return parameters
D. ☐ reference parameters only

Q17. Which statement about an **inline** function is correct?

- A. ☐ The compiler is forced to substitute the function body at every call.
B. ☐ **inline** is a request to the compiler, not a guarantee.
C. ☐ An inline function cannot return a value.
D. ☐ Only member functions can be inline.

Q18. Which pair is a valid example of function overloading?

- A. ☐ `int area(int); double area(int);`
- B. ☐ `int area(int); int area(double);`
- C. ☐ `int area(int); int area(int);`
- D. ☐ `int area(int x); int area(int y);`

Q19. Why is the following not valid overloading?

```
int calculate(int x);
double calculate(int x);
```

- A. ☐ Parameter names must be different.
- B. ☐ Functions cannot be overloaded by return type alone.
- C. ☐ A function name may appear only once in a program.
- D. ☐ `double` cannot be used as a return type.

Q20. Which declaration follows the C++ trailing-default rule?

- A. ☐ `void f(int a = 1, int b);`
- B. ☐ `void f(int a, int b = 2);`
- C. ☐ `void f(int a = 1, int b, int c = 3);`
- D. ☐ `void f(int a = 1, int b, int c);`

Q21. Consider:

```
void print(int x);
void print(int x, int y = 0);
```

What happens for the call `print(5);`?

- A. ☐ Only the first overload matches.
- B. ☐ Only the second overload matches.
- C. ☐ The call is ambiguous.
- D. ☐ Neither overload matches.

Q22. Which statement about C-style variadic functions is correct?

- A. ☐ They are fully type-safe for every extra argument.
- B. ☐ They can accept a variable number of arguments but require careful handling of argument types.
- C. ☐ They can be used only as member functions.
- D. ☐ They automatically store extra arguments in a `vector`.

Q23. What type is deduced for `x`?

```
auto x = 25;
```

- A. ☐ `int`
- B. ☐ `double`
- C. ☐ `char`
- D. ☐ `int&`

Q24. Which declaration is invalid?

- A. ☐ `auto x = 10;`
- B. ☐ `auto y = 3.5;`
- C. ☐ `auto name = "C++";`
- D. ☐ `auto value;`

Q25. What is the output?

```
int x = 5;
auto &r = x;
r = 12;
cout << x;
```

- A. ☐ 5
- B. ☐ 12
- C. ☐ 0
- D. ☐ Compilation error

Q26. Which loop safely reads each element without copying it and prevents modification through the loop variable?

- A. ☐ `for (auto x : values)`
- B. ☐ `for (auto& x : values)`
- C. ☐ `for (const auto& x : values)`
- D. ☐ `for (auto* x : values)`

Q27. What is the final content of the array?

```
int a[] = {1, 2, 3};
for (auto x : a) {
    x = x * 10;
}
```

- A. ☐ `{10, 20, 30}`
- B. ☐ `{1, 2, 3}`
- C. ☐ `{0, 0, 0}`
- D. ☐ The code does not compile.

Q28. Choose the correct loop to multiply every element of `values` by 2.

- A. ☐ `for (auto x : values) x *= 2;`
- B. ☐ `for (auto& x : values) x *= 2;`
- C. ☐ `for (const auto& x : values) x *= 2;`
- D. ☐ `for (auto x = values) x *= 2;`

Q29. What is printed?

```
int a[] = {2, 4, 6};  
for (auto x : a)  
    cout << x << " ";
```

- A. ☐ 0 1 2
- B. ☐ 2 4 6
- C. ☐ 6 4 2
- D. ☐ 2 2 2

Q30. Which situation most strongly favours a tradi-

tional indexed **for** loop over a range-based **for** loop?

- A. ☐ You only need to print every element in order.
- B. ☐ You need the numeric index of each element during traversal.
- C. ☐ You want read-only access to every element.
- D. ☐ You want to avoid writing the element type explicitly.

Score:

Check Score

Reset Form

Automatic scoring requires a PDF viewer that supports Acrobat JavaScript (recommended: Adobe Acrobat Reader desktop).

— **End of Test** —